TWO POINTERS

DSA Revision Notes — Easy Hinglish + Pseudocode

Kaise use karna hai: Har question mein simple meaning, direct approach, main-logic pseudocode, TC/SC aur ek common mistake hai. Full Java class ya main() boilerplate nahi diya hai.

Pointer patterns yaad rakho

Pattern	Use
Opposite direction	left start par, right end par; pair/triplet sums ya palindrome.
Same direction / write pointer	Ek pointer scan kare, doosra next correct position track kare.
Three pointers / DNF	0, 1, 2 jaise 3 categories ko in-place partition karna.
Greedy boundary scan	Partition Labels; related technique, classic two-pointer nahi.

Note: Tumne explicitly mention kiye questions aur pehle discuss kiye gaye problems ko combine karke ye 14 rakhe hain. Two Sum I HashMap pattern hai; sorted two-pointer version Two Sum II hai.

1. Two Sum (LeetCode 1) — HashMap

Simple meaning: Normal Two Sum I sorted hona guarantee nahi hota, isliye standard solution HashMap hai. Har number ke liye target banane wala complement check karo.

Approach — kya karna hai

1. 1. Map mein number aur index store karo.

2. 2. Har nums[i] ke liye complement = target - nums[i].

3. 3. Complement pehle map mein ho toh dono indices return.

4. 4. Nahi toh current number/index map mein add karo.

Main logic ka pseudocode

```
map = empty map
for i = 0 to n-1:
complement = target - nums[i]
if complement in map: return [map[complement], i]
map[nums[i]] = i
```

TC: O(n) **SC:** O(n)

Yaad rakhna: Ye classic two-pointer nahi; hashing pattern hai. Sorted version Two Sum II hai.

2. Two Sum II (LeetCode 167)

Simple meaning: Sorted array mein left sabse chhote aur right sabse bade number par rakho. Sum ke hisaab se pointer move karo.

Approach — kya karna hai

1. 1. left = 0, right = n-1.

2. 2. Sum target ke equal ho toh answer mil gaya.

3. 3. Sum chhota ho toh left++.

4. 4. Sum bada ho toh right--.

Main logic ka pseudocode

```
left = 0; right = n-1
while left < right:
sum = nums[left] + nums[right]
if sum == target: return answer
else if sum < target: left++
else: right--
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Sorted order ki wajah se left++ sum badhata hai aur right-- sum ghatata hai.

3. Valid Palindrome (LeetCode 125)

Simple meaning: String ko dono ends se compare karo. Non-alphanumeric characters ignore karo aur case ko ignore karne ke liye lowercase compare karo.

Approach — kya karna hai

1. 1. left aur right ends par rakho.

2. 2. Letter/digit nahi ho toh pointer skip kare.

3. 3. Lowercase characters compare karo.

4. 4. Mismatch ho toh false; match ho toh dono pointers andar.

Main logic ka pseudocode

```
left = 0; right = s.length-1
while left < right:
skip non-letter/digit at left and right
if lowercase(s[left]) != lowercase(s[right]): return false
left++; right--
return true
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Punctuation aur spaces ignore karne ka instruction ho toh unhe skip karna.

4. Valid Palindrome II (LeetCode 680)

Simple meaning: Ek character tak delete kar sakte ho. First mismatch par left character skip karke check karo ya right character skip karke.

Approach — kya karna hai

1. 1. Ends se compare karo.

2. 2. First mismatch par do possible ranges check karo.

3. 3. Dono mein se koi ek palindrome ho toh true.

4. 4. Helper given range ko check kare.

Main logic ka pseudocode

```
left = 0; right = s.length-1
while left < right:
if s[left] != s[right]:
return isPalindrome(left+1, right) OR isPalindrome(left, right-1)
left++; right--
return true
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Mismatch par sirf ek deletion allowed hai; dono sides skip karke aage continue nahi karna.

5. Move Zeroes (LeetCode 283)

Simple meaning: Non-zero values ka order same rakho aur zeroes end mein le jao. Extra array nahi chahiye.

Approach — kya karna hai

1. 1. j next non-zero ki placement hai.

2. 2. i poora array scan kare.

3. 3. Non-zero mile toh nums[i] aur nums[j] swap.

4. 4. Swap ke baad j++.

Main logic ka pseudocode

```
j = 0
for i = 0 to n-1:
if nums[i] != 0:
swap(nums[i], nums[j])
j++
```

TC: O(n) **SC:** O(1)

Yaad rakhna: j next non-zero position ko track karta hai.

6. Remove Duplicates from Sorted Array (LeetCode 26)

Simple meaning: Sorted array mein duplicates adjacent hote hain. Unique values ko start mein compact karo aur unique count return karo.

Approach — kya karna hai

1. 1. i last unique element ki position.

2. 2. j = 1 se scan kare.

3. 3. nums[j] != nums[i] ho toh i++ aur nums[i] = nums[j].

4. 4. Return i+1.

Main logic ka pseudocode

```
if n == 0: return 0
i = 0
for j = 1 to n-1:
if nums[j] != nums[i]:
i++
nums[i] = nums[j]
return i+1
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Input sorted hona zaroori hai, tabhi duplicates adjacent milte hain.

7. Squares of a Sorted Array (LeetCode 977)

Simple meaning: Negative numbers ka square bhi bada ho sakta hai. Largest square dono ends mein se aayega, isliye result ko end se fill karo.

Approach — kya karna hai

1. 1. left=0, right=n-1, k=n-1.

2. 2. Original values ke absolute values compare karo.

3. 3. Bada absolute value square karke result[k] mein rakho.

4. 4. Used pointer move karo aur k--.

Main logic ka pseudocode

```
left = 0; right = n-1; k = n-1
while left <= right:
if abs(nums[left]) >= abs(nums[right]):
result[k] = nums[left]^2; left++
else:
result[k] = nums[right]^2; right--
k--
return result
```

TC: O(n) **SC:** O(n) output; O(1) auxiliary

Yaad rakhna: Pehle input square karke sort mat karo; woh O(n log n) ho jayega.

8. Sort Colors (LeetCode 75)

Simple meaning: Array mein sirf 0, 1, 2 hain. low, mid, high se 0 left, 1 middle, 2 right mein arrange karo.

Approach — kya karna hai

1. 1. low=0, mid=0, high=n-1.

2. 2. 0 mile: low-mid swap; low++, mid++.

3. 3. 1 mile: mid++.

4. 4. 2 mile: mid-high swap; high--, mid same.

Main logic ka pseudocode

```
low=0; mid=0; high=n-1
while mid <= high:
if nums[mid] == 0: swap(nums[low],nums[mid]); low++; mid++
else if nums[mid] == 1: mid++
else: swap(nums[mid],nums[high]); high--
```

TC: O(n) **SC:** O(1)

Yaad rakhna: 2 par mid++ nahi karte, kyunki high se aaya element abhi check karna hai.

9. Container With Most Water (LeetCode 11)

Simple meaning: Area = width × shorter height. Har pair check nahi karna; shorter wall wale pointer ko move karo.

Approach — kya karna hai

1. 1. left=0, right=n-1.

2. 2. Area calculate karke maximum update karo.

3. 3. Jis side ki height chhoti hai, us pointer ko andar lao.

4. 4. Jab pointers mil jaayein, stop.

Main logic ka pseudocode

```
left=0; right=n-1; best=0
while left < right:
area = (right-left) * min(height[left], height[right])
best = max(best, area)
if height[left] < height[right]: left++
else: right--
return best
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Shorter wall area ko limit karti hai, isliye us side ko move karte hain.

10. Trapping Rain Water (LeetCode 42)

Simple meaning: Har index par paani = min(leftMax,rightMax) - current height. Two pointers se maxima maintain kar sakte hain.

Approach — kya karna hai

1. 1. left/right ends, leftMax/rightMax=0.

2. 2. left height <= right height ho toh left side process.

3. 3. Warna right side process.

4. 4. Relevant max update karke trapped water add karo; pointer move karo.

Main logic ka pseudocode

```
left=0; right=n-1; leftMax=0; rightMax=0; water=0
while left < right:
if height[left] <= height[right]:
leftMax=max(leftMax,height[left])
water += leftMax-height[left]; left++
else:
rightMax=max(rightMax,height[right])
water += rightMax-height[right]; right--
return water
```

TC: O(n) **SC:** O(1)

Yaad rakhna: Jo side process kar rahe ho, us side ka maximum use karke water count karo.

11. 3Sum (LeetCode 15)

Simple meaning: Teen numbers ka sum zero. Ek number fix karo aur baaki do sorted array mein two pointers se find karo. Unique triplets chahiye.

Approach — kya karna hai

1. 1. Array sort karo.

2. 2. i fix; left=i+1, right=n-1.

3. 3. Sum 0 ho toh triplet add, pointers move, duplicates skip.

4. 4. Sum negative ho toh left++; positive ho toh right--.

Main logic ka pseudocode

```
sort(nums)
for i = 0 to n-3:
skip duplicate i
left=i+1; right=n-1
while left < right:
sum=nums[i]+nums[left]+nums[right]
if sum == 0: add triplet; left++; right--; skip duplicates
else if sum < 0: left++
else: right--
```

TC: O(n^2) **SC:** O(1) auxiliary; output excluded

Yaad rakhna: Unique triplets ke liye i, left aur right ke duplicates skip karna.

12. 3Sum Closest (LeetCode 16)

Simple meaning: Teen numbers ka sum target ke closest chahiye. Sirf closest sum return karna hai, triplets ki list nahi.

Approach — kya karna hai

1. 1. Sort; closest ko first three numbers ke sum se initialize karo.

2. 2. i fix; left=i+1, right=n-1.

3. 3. Current sum target se kitna door hai, closest se compare karo.

4. 4. Sum chhota ho toh left++; bada ho toh right--; exact target par return.

Main logic ka pseudocode

```
sort(nums)
closest = nums[0]+nums[1]+nums[2]
for i = 0 to n-3:
left=i+1; right=n-1
while left < right:
sum=nums[i]+nums[left]+nums[right]
if abs(sum-target) < abs(closest-target): closest=sum
if sum == target: return sum
else if sum < target: left++
else: right--
return closest
```

TC: O(n^2) **SC:** O(1) auxiliary; sort implementation separate

Yaad rakhna: Duplicates skip karna correctness ke liye required nahi, kyunki sirf sum return hota hai.

13. 4Sum (LeetCode 18)

Simple meaning: Chaar numbers ka sum target. 3Sum jaisa, lekin i aur j dono fix karke baaki pair search karte hain.

Approach — kya karna hai

1. 1. Sort karo.

2. 2. i fix, phir j fix.

3. 3. left=j+1, right=n-1.

4. 4. Sum chhota ho toh left++; bada ho toh right--.

5. 5. Match par answer add, pointers move aur duplicates skip. Sum ke liye long use karo.

Main logic ka pseudocode

```
sort(nums)
for i = 0 to n-4:
skip duplicate i
for j = i+1 to n-3:
skip duplicate j
left=j+1; right=n-1
while left < right:
sum=nums[i]+nums[j]+nums[left]+nums[right]
if sum == target: add quadruplet; left++; right--; skip duplicates
else if sum < target: left++
else: right--
```

TC: O(n^3) **SC:** O(1) auxiliary; output excluded

Yaad rakhna: Do fixed indices chahiye. Long sum overflow se bachata hai.

14. Partition Labels (LeetCode 763)

Simple meaning: String ko maximum parts mein baanto taaki har character sirf ek part mein aaye. Character ka last occurrence usi part ke andar hona chahiye.

Approach — kya karna hai

1. 1. Har character ka last index store karo.

2. 2. start=0, end=0.

3. 3. Scan karte hue end=max(end,lastIndex[current char]).

4. 4. i==end par partition complete; length add karo aur next start set karo.

Main logic ka pseudocode

```
last = last index of every character
start=0; end=0; sizes=[]
for i = 0 to s.length-1:
end=max(end,last[s[i]])
if i == end:
sizes.add(end-start+1)
start=i+1
return sizes
```

TC: O(n) **SC:** O(1) fixed alphabet; otherwise O(k)

Yaad rakhna: Ye classic two-pointer question nahi; greedy boundary scan hai, but related list mein include kiya hai.